Week 1 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Hints and solutions

Instructor guide

Reference sheets

Download all code

1 · Staged hints

LeetCode 1: Two Sum · Easy

New exercise

H1. Which value would complete the current value?

H2. When should an index be recorded to avoid using it twice?

H3. Store earlier value-to-index pairs; look up target-value before recording the current value.

1 · Correctness and cost

At loop entry, the map contains only earlier positions. At the later position of the unique answer, its partner is present. This reuses information instead of checking all $O(n^2)$ pairs.

n is the number of elements. Expected O(n) time and O(n) auxiliary space; adversarial hash collisions can give $O(n^2)$ time.

1 · Counterexamples and checks

Recording first can reuse one index on [3,3], target=6.

```
nums=[4,8,3], target=7 → [0,2]
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

1 · Implementation 1/1

H4. Concatenate these portions to recover the complete implementation in the code bundle.

```
def two_sum(nums, target):
    seen = {}
    for index, value in enumerate(nums):
        if target - value in seen:
            return [seen[target - value], index]
        seen[value] = index
    return []  # Defensive extension: official inputs have one answer.
```

219 · Staged hints

LeetCode 219: Contains Duplicate II · Easy

New exercise

H1. Which occurrence of a value should be retained?

H2. Can an older occurrence be closer than the latest one?

H3. Retain the latest index per value; compare distance, then update.

219 · Correctness and cost

The latest occurrence minimizes distance to the current index. If it is too far, all older occurrences are too far. Updating preserves this property.

n is length. Expected O(n) time, O(n) space; $O(n^2)$ worst-case hashing time.

219 · Counterexamples and checks

Keeping only the first position misses the last pair in [4,7,4,4], k=1. No distinct-index pair qualifies when k=0.

```
nums=[4,7,4,4], k=1 → True
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

219 · Implementation 1/1

H4. Concatenate these portions to recover the complete implementation in the code bundle.

```
def nearby_duplicate(nums, k):
    latest = {}
    for index, value in enumerate(nums):
        if value in latest and index - latest[value] <= k:
            return True
        latest[value] = index
    return False
```

167 · Staged hints

LeetCode 167: Two Sum II - Input Array Is Sorted · Medium

New exercise

H1. If the endpoint sum is too small, which candidates are all too small?

H2. Does the elimination remain valid without sorting?

H3. While left<right, move left inward for a small sum and right inward for a large sum.

167 · Correctness and cost

If the sum is small, no partner at or below right works with left. If large, no partner at or above left works with right. Each elimination preserves any answer.

n is length. Total pointer movements O(n), time O(n), auxiliary space O(1).

167 · Counterexamples and checks

Outside the sorted contract, [3,1,4], target=5 defeats this rule. left==right reuses one element.

```
numbers=[1,4,6,9], target=10 → [1,4]
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

167 · Implementation 1/1

H4. Concatenate these portions to recover the complete implementation in the code bundle.

```
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

15 · Staged hints

LeetCode 15: 3Sum · Medium

New exercise

H1. What remains after fixing one value?

H2. Does fixing the same value again create a new triplet?

H3. Sort a copy, fix the first value, and use two pointers on the suffix; skip repeated fixed values and matched pair values.

15 · Correctness and cost

Every ordered triplet is examined when its smallest value is fixed. Sorted elimination is valid; skipping duplicates removes repeated value combinations only.

n is length, z the output count. $O(n^2)$ time, O(n) copy/sort space, O(z) output.

15 · Counterexamples and checks

[0,0,0,0] yields one triplet. Equal values at distinct positions remain allowed.

```
nums=[-2,0,1,1,2] → [[-2,0,2],[-2,1,1]]
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

15 · Implementation 1/2

H4. Concatenate these portions to recover the complete implementation in the code bundle.

```
def three_sum(nums):
    values = sorted(nums)
    result = []
    for index in range(len(values) - 2):
        if index and values[index] == values[index - 1]:
            continue
        left, right = index + 1, len(values) - 1
        while left < right:
            total = values[index] + values[left] + values[right]
            if total < 0:
                left += 1
```

15 · Implementation 2/2

H4. Concatenate these portions to recover the complete implementation in the code bundle.

```
            elif total > 0:
                right -= 1
            else:
                result.append([values[index], values[left], values[right]])
                left += 1
                right -= 1
                while left < right and values[left] == values[left - 1]:
                    left += 1
                while left < right and values[right] == values[right + 1]:
                    right -= 1
    return result
```

Execution and sources

```
cd code
python demo_01.py
python demo_02.py
python verify.py
```

Completed implementations are in solutions.py; independent work is in student_week_01.py. On LeetCode, adapt each function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).